“DESIGN OF A TEMPERATURE DATA LOGGING SYSTEM”

Academic Internship report submitted in partial fulfilment of the requirement for the degree of

Bachelor of Technology in Electrical Engineering

Submitted by:

Hrishikesh Kalita (240650003007)

Sachidananda Mukherjee (230610003080)

Department of Electrical Engineering

Assam Engineering College,

Jalukbari, 781013

August 2025

ACKNOWLEDGEMENT

We would like to express our sincere appreciation to our esteemed guide and mentor from the Indian Institute of Technology Guwahati (IIT Guwahati) for their unwavering dedication, expertise, and invaluable support throughout the duration of this project. Their profound knowledge and insightful guidance played a pivotal role in shaping our ideas, streamlining our efforts, and overcoming the technical challenges we encountered during the development of the data logging system. Their patience, encouragement, and open-mindedness have been a constant source of motivation and inspiration.

We would also like to express our heartfelt gratitude to the Department of Electrical Engineering, Assam Engineering College for providing us with the necessary academic resources, tools, and a conducive learning environment. The continuous support and encouragement from our faculty members have been instrumental in enhancing our technical skills and deepening our understanding of embedded systems and IoT applications.

We further extend our sincere thanks to the academic staff and administrative team for their assistance and cooperation, which ensured that we had all the required facilities available during the internship period. We are also grateful to the authors and researchers whose publications, articles, and reference materials helped us understand the scientific and practical aspects of thermal and environmental data logging in cold storage systems.

In conclusion, this project would not have been possible without the collective efforts and contributions of everyone mentioned above. We are truly grateful and humbly acknowledge their role in shaping our journey toward academic and technical growth.

ABSTRACT

This project focuses on the development of a data logging system to support the efficient design of cold storage facilities for fruits and vegetables. The system is designed to monitor and record temperature as a function of depth and time below the ground, as well as temperature and relative humidity (T + RH) above ground. Accurate, long-term environmental data is critical for optimizing storage conditions and minimizing post-harvest losses. Using microcontrollers and sensors, the system collects data continuously and stores it for analysis. The recorded data will serve as a foundation for understanding thermal behaviour over time and depth, ultimately aiding in the scientific planning of low-cost, energy-efficient cold storage solutions. This work was carried out as part of a one-month academic internship at IIT Guwahati.

TABLE OF CONTENTS

	Page No:
ACKNOWLEDGEMENT	2
ABSTRACT	3
CHAPTER 1: INTRODUCTION	6
1.1 Background and motivation	6
1.2 Problem Statement	7
1.3 Objectives of the project	8
1.4 Internship scope	9
CHAPTER 2: COMPONENTS USED	10
2.1 List of hardware components	
2.2 Sim800l Module	
2.3 Wroom Esp32	
2.4 Comparators, optocouplers and diode	
2.5 Software tools used	
CHAPTER 3: SYSTEM DESIGN AND IMPLEMENTATION	18
3.1 Block diagram	
3.2 Connection diagram	
3.3 Solar power integration	
3.4 Battery monitoring system	
3.5 Microcontroller configuration	
3.6 Data transmission to cloud	
3.7 Final mounting and setup	

Page No:

CHAPTER 4: RESULTS AND DISCUSSIONS 37

4.1 Sensor performance

4.2 Overall system validation

CHAPTER 5: CONCLUSION AND FUTURE WORK 41

5.1 Conclusion

5.2 Limitations

5.3 Future Enhancements

REFERENCE 44

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

India is one of the top producers of fruits and vegetables in the world. But even with such a large harvest, a lot of this food goes to waste every year. This happens mainly because there aren't enough good storage facilities after the crops are harvested. Fruits and vegetables spoil quickly, so they need to be kept in cold storage to stay fresh for longer.

Cold storage helps in slowing down spoilage and keeps the produce fresh until it reaches the market. However, most of the cold storage units in India are old or not built in the right way. Many of them don't consider the local weather or environmental conditions. Also, they don't track important things like temperature and humidity in real-time or over a long period. Because of this, it becomes hard to build or improve cold storage systems that are energy-saving and suitable for different regions.

Monitoring temperature, humidity, and other environmental factors is very important to make cold storage more effective. Without proper data, farmers and storage operators cannot know if the storage conditions are right. Poor conditions can lead to food spoilage and higher electricity use, which increases costs and harms the environment.

So, there is a strong need to collect and use real-time and long-term data from cold storage units. This will help in designing better systems that save energy and reduce food

waste. Having the right data can also help in building cold storage that works well for specific crops and local weather conditions, leading to better profits for farmers and a more reliable food supply for everyone.

1.2 PROBLEM STATEMENT

Conventional cold storage units usually run without any kind of system to monitor or respond to environmental changes. In most cases, they do not have sensors or tools to check and record important conditions such as soil temperature at different depths, the temperature of the air around the unit (ambient temperature), or the level of moisture in the air (relative humidity) over time. Because of this, the storage environment often becomes unstable or inconsistent, which can cause fruits, vegetables, and other perishable items to spoil faster than expected.

When cold storage systems operate without proper feedback or real-time monitoring, they may use more electricity than necessary. For example, cooling might continue even when it's not needed, or it might fail to adjust to sudden temperature or humidity changes. This not only increases energy costs but also affects the quality and safety of the stored produce. Over time, these issues lead to significant losses in both food and money.

Another major problem is the lack of data. Without proper records of environmental conditions, it becomes very difficult to study how different factors affect cold storage performance. Designers and engineers cannot improve the system or make it suitable for specific crops, regions, or climate conditions without accurate, long-term data. Every crop has different

needs, and the storage environment must be carefully managed to meet those needs. But if no data is collected, there is no way to make smart decisions or improvements.

This project aims to solve these problems by developing and installing a data logging system. The goal is to continuously monitor and record important environmental conditions, such as soil and air temperature, humidity, and possibly other factors, depending on the need. The data will be stored and used to study how the cold storage performs under different conditions. With this information, it will be possible to create better, more efficient storage systems that are tailored to specific crops and local environments. In the long run, this can help reduce food spoilage, save energy, and support farmers by improving post-harvest storage.

1.3 OBJECTIVES OF THE PROJECT

The objective of this project was to design and implement a data logging system capable of continuously collecting environmental data relevant to cold storage performance. Specifically, the system monitors temperature as a function of depth and time below ground, as well as temperature and relative humidity (T + RH) above ground. The data collected over extended periods will serve as a scientific basis for optimizing cold storage structure, insulation, and placement. The goal is to enable the development of intelligent, cost-effective, and regionally adaptable storage solutions for perishable agricultural products.

1.4 INTERNSHIP SCOPE

During the one-month academic internship at IIT Guwahati, we undertook the complete design and development of a data logging system from the ground up. The process involved identifying key storage and performance requirements, selecting appropriate sensors (such as the DHT22 for temperature and humidity and underground moisture sensors), and choosing suitable microcontrollers (ESP32) to ensure efficient data acquisition and processing. We integrated all hardware and software components into a cohesive, functioning prototype. The system was programmed to collect, store, and transmit data continuously, ensuring reliable environmental monitoring. Initial data sets were retrieved, processed, and analysed to understand environmental behaviour over time and assess the overall performance of the logger. The internship offered hands-on experience with embedded systems, real-time data collection, sensor calibration, and data-driven design thinking, significantly enhancing our practical understanding of IoT-based monitoring solutions.

CHAPTER 2
COMPONENTS USED

2.1 LIST OF HARDWARE COMPONENTS

There are5 major components of data logging system:

1) Devices and sensors

   a) DHT22 Sensor

   The DHT22 is a digital sensor capable of measuring both temperature and relative humidity.

Fig: DHT22 sensor

It provides high accuracy, long-term stability, and low power consumption, making it suitable for agricultural and environmental monitoring applications. It is used in this project to measure ambient temperature and humidity (T+RH) above ground.

   b) SIM800L GSM Module

   The SIM800L is a GSM/GPRS communication module that enables the system to send sensor data via SMS or upload to the cloud using mobile networks (2G).

Fig: SIM800l module

It serves as an alternative to Wi-Fi in remote areas where internet access is not available.

c) Soil Temperature Sensors

Temperature probes DS18B20 were embedded at various depths below the ground to record subsurface thermal gradients over time.

Fig: DS18B20 sensor

These readings are crucial for analysing the thermal behaviour of soil, which plays a major role in designing passive cold storage systems.

d) ESP32 Microcontroller

The ESP32 is a powerful microcontroller with built-in Wi-Fi and Bluetooth capabilities.

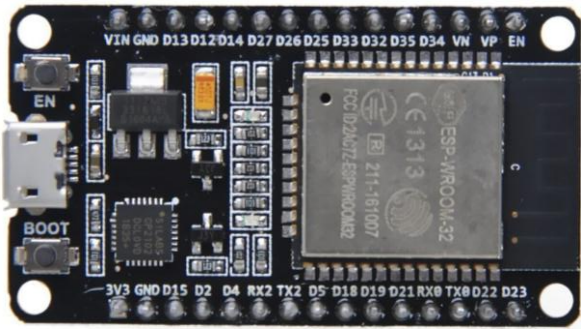


Fig: Wroom ESP 32

It acts as the core processing unit of the system, responsible for interfacing with sensors, timestamping data, storing it locally, and uploading it to the cloud. It was chosen for its low power consumption and IoT compatibility.

e) Real-Time Clock (RTC) Module

An RTC module (e.g., DS3231) was used to maintain accurate time and date information for timestamping each data entry, even during power outages or reboots.

Fig; DS3231 RTC module

2) Solar power system

a) Solar Panel: Converts sunlight into electrical energy.

Fig: Solar Panel

b) Charge Controller: Manages charging of the battery and prevents overcharging or deep discharge.

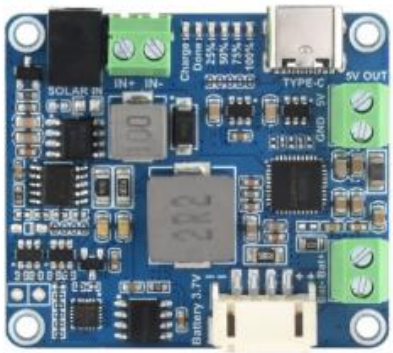


Fig: Solar charge controller

c) Battery: Typically, two 3.7V Li-ion is used to store power for nighttime operation.

Fig: ICR18650 Li-ion battery

d) Buck converter: Ensure appropriate voltage levels for components—especially critical for the SIM800L (which requires 3.7V–4.2V).

Fig: LM2596 Buck converter

3) Battery monitoring system:

To ensure reliable power management for the solar-powered data logging system, a battery monitoring circuit was integrated.

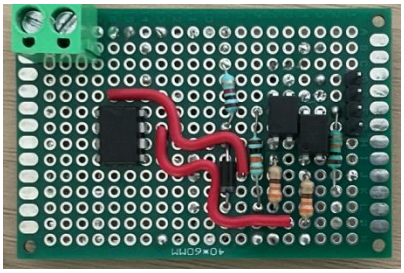


Fig: Battery monitoring circuit

This system continuously monitors the battery voltage level and provides a logic-level signal when the battery drops below a predefined threshold, helping prevent deep discharge and ensuring safe operation of the system.

2.2 SIM800L MODULE

The SIM800L is a compact GSM/GPRS module that enables communication over a cellular network. In this project, the SIM800L plays a crucial role in transmitting real-time data and alerts from the cold storage system to the user's mobile device via SMS. It is used to send important notifications such as abnormal temperature levels or low battery status, ensuring timely intervention. The module operates on standard AT commands and communicates with the ESP32 microcontroller through UART serial communication. Its low power consumption and compatibility with various GSM networks make it ideal for remote monitoring applications, especially in areas without reliable internet connectivity. Proper antenna placement and power regulation were ensured to maintain stable signal strength and communication reliability.

2.3 WROOM ESP32

The ESP32 is a powerful, low-cost microcontroller developed by Espressif Systems, known for its integrated Wi-Fi and Bluetooth capabilities, making it an ideal choice for IoT applications. In this project, the ESP32 acts as the central processing unit, responsible for reading temperature data from two DS18B20 digital sensors and one DHT22 sensor. It also receives battery level information from a comparator-based detection circuit. After collecting the sensor data, the ESP32 processes and formats it into a JSON object and sends it to the

ThingSpeak cloud platform through a SIM800L GSM module using AT commands over UART communication.

The ESP32 offers dual-core processing, a clock speed of up to 240 MHz, multiple GPIO pins, and inbuilt ADC channels, all of which are utilized effectively in this system. Its ability to handle multiple sensors, perform real-time data processing, and support various communication protocols makes it a highly suitable controller for this cold storage monitoring project. Overall, the ESP32 was selected for its performance, versatility, ease of programming, and suitability for field-based IoT deployments where wireless data transmission is required.

2.4 COMPARATORS, OPTOCOUPLERS AND DIODE

The comparator is used to compare the battery voltage against predefined reference voltage levels set using a voltage divider network. Depending on whether the battery voltage is below or above a certain threshold, the comparator outputs a HIGH or LOW signal. This output is then passed through an optocoupler, which electrically isolates the sensing circuit from the ESP32 microcontroller. This isolation helps protect the microcontroller from potential voltage spikes or fluctuations in the battery circuit, enhancing the system's overall reliability and safety.

Additionally, diodes are used in the circuit to prevent reverse current flow and protect sensitive components from incorrect polarity connections. The combination of these components provides a three-level battery status indication—typically categorized as LOW, MEDIUM, and HIGH—based

on the voltage levels. Although this approach does not provide exact voltage readings, it offers a cost-effective and reliable method for basic battery health indication. The integration of these components plays a crucial role in maintaining safe operation and ensuring the ESP32 receives clean, isolated signals representing battery condition in real time.

2.5 SOFTWARE TOOLS USED

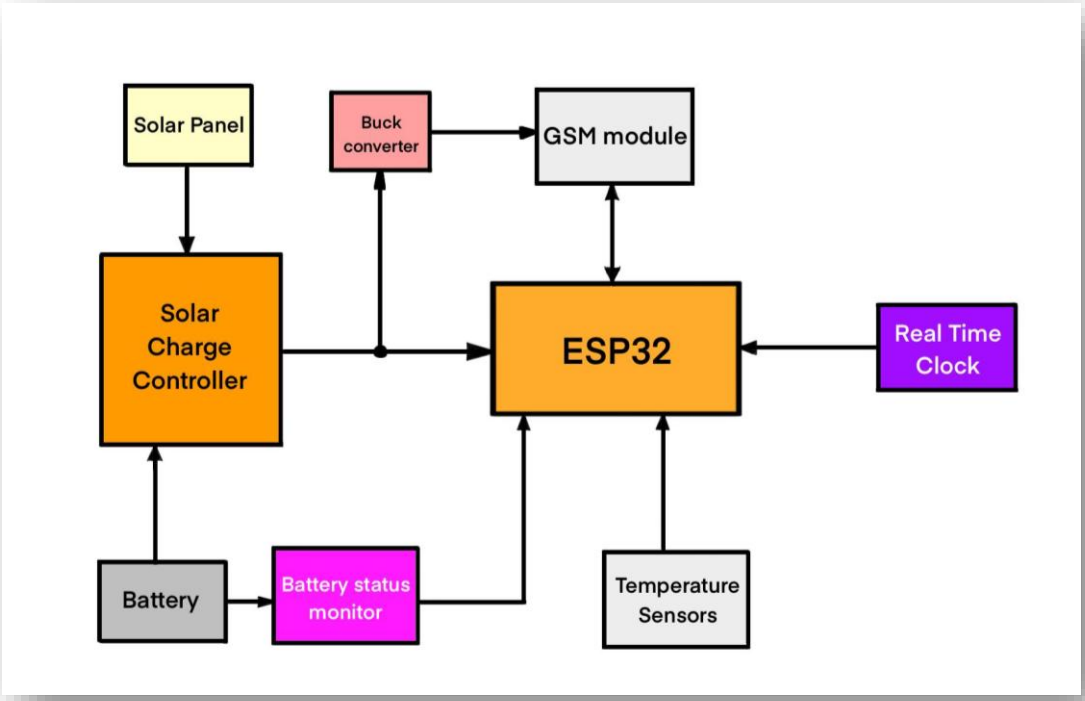
In this project, two primary software tools were used:

a) Arduino IDE: The Arduino IDE was chosen for programming the ESP32 microcontroller due to its simplicity, wide community support, and compatibility with various libraries needed for sensor interfacing and GSM communication. It allowed us to write, compile, and upload code efficiently to control the system's logic and communication flow.

b) ThinkSpeak: It was used for real-time data visualization and storage. It provided an easy way to monitor sensor data remotely through graphical dashboards, making it useful for observing temperature trends or battery status over time. Together, these tools enabled seamless development, communication, and data monitoring for the cold storage system.

CHAPTER 3
SYSTEM DESIGN AND IMPLEMENTATION

3.1 BLOCK DIAGRAM



The block diagram illustrates the complete working structure of the IoT-based cold storage monitoring system. The system is designed for off-grid applications where real-time monitoring, energy efficiency, and remote accessibility are critical. At the centre of the system is the ESP32 microcontroller, which acts as the brain of the setup. It is responsible for collecting data, processing information, and controlling communication between various modules.

The temperature sensors are connected to the ESP32 and are used to monitor the internal conditions of the cold storage

environment. These sensors continuously measure the temperature and sendanalogsignals to the ESP32, which processes the data and determines whether the values fall within the safe range.

A Battery Status Monitor is integrated into the circuit to continuously check the charge level of the battery. It plays a crucial role in ensuring the system remains operational, especially in remote or off-grid locations. The ESP32 reads the battery status and can trigger alerts if the battery voltage drops below a certain threshold.

To maintain accurate records and to enable time-based analysis, a Real-Time Clock (RTC) module is connected to the ESP32. This module provides precise time and date data, which is essential for logging sensor readings and scheduling communication events.

For wireless communication, a SIM800L GSM module is employed. It allows the system to send SMS alerts to the user's mobile phone whenever there is a significant temperature deviation or battery drop. The GSM module communicates with the ESP32 through UART serial communication, and it operates on a lower voltage, requiring the use of a Buck Converter to safely step down the voltage from the battery or power supply.

Power is supplied to the system primarily through a solar panel, making it suitable for rural or off-grid installations. The solar panel charges the battery via a Solar Charge Controller, which regulates the voltage and current coming from the panel to

protect the battery from overcharging or deep discharge. The solar charge controller also supplies power to the ESP32 and other modules, ensuring the entire system is self-sustainable and reliable in long-term operations.

3.2 CONECTION DIAGRAM

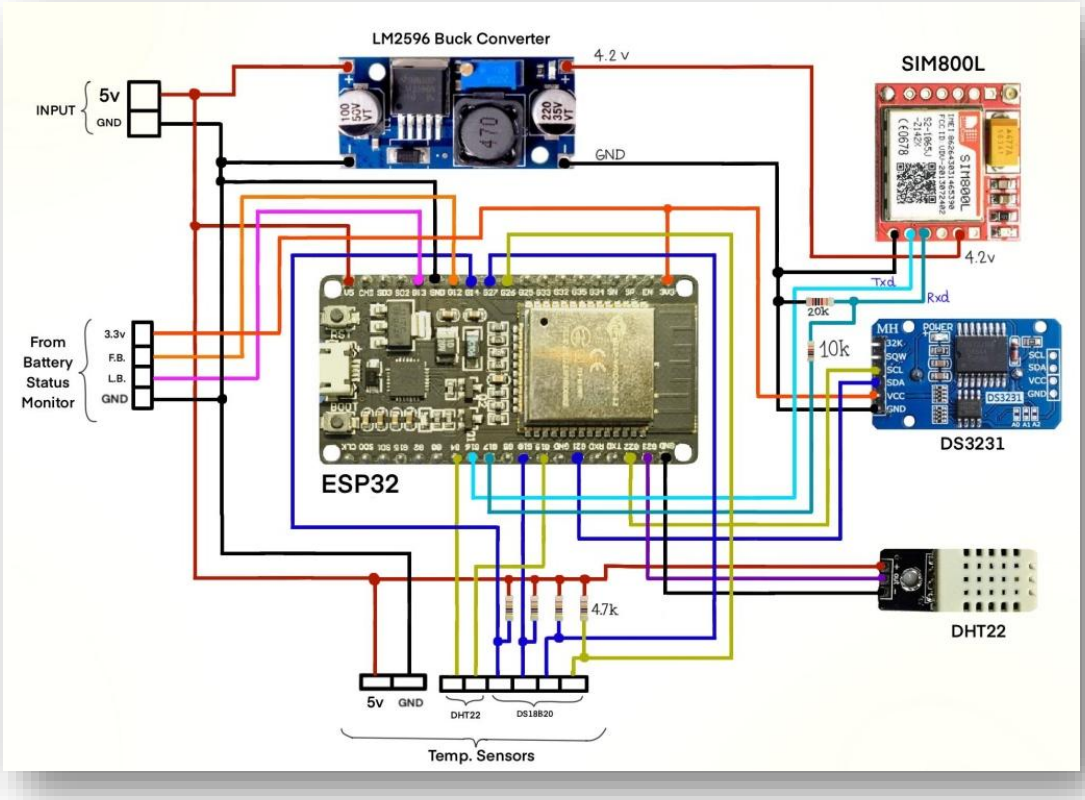


Fig: Connection diagram of data logger

The circuit diagram provides a detailed view of how the hardware components are interconnected in the controller unit. Unlike the conceptual overview in the block diagram, this circuit illustration focuses on the specific pin-to-pin connections, voltage levels, and the practical arrangement of components on the microcontroller.

At the centre is the ESP32 development board, which interfaces with multiple modules and sensors. The SIM800L GSM module, operating at 4.2V, is powered through a buck converter (LM2596) that steps down the input 5V supply. The GSM module's TX and RX pins are connected to the ESP32's UART pins, allowing for reliable SMS communication. Pull-down resistors (10kΩ and 20kΩ) are included to stabilize the logic levels.

The DS3231 RTC module is connected using the I2C interface (SDA and SCL lines), which allows the ESP32 to fetch precise time data for timestamping temperature readings and battery status logs. This module is powered through a 3.3V or 5V line, depending on the ESP32's output and the module version.

The system includes two temperature sensing interfaces; The DHT22 sensor, which provides temperature and humidity readings, is connected to a GPIO pin and requires a 10kΩ pull-up resistor on the data line.

The DS18B20 sensor (in multiple instances) is used for more precise or distributed temperature sensing, especially in colder zones. It connects through a 1-Wire protocol and uses a 4.7kΩ resistor for stable data communication.

A custom-built battery status monitor circuit outputs three distinct logic signals: Full, Low, and Danger. These signals are connected to three digital input pins of the ESP32, which reads them as HIGH or LOW values to determine battery health.

Based on this, the ESP32 can issue warnings or trigger system behaviour (e.g., disabling power-heavy operations).

All modules share a common ground (GND), ensuring a stable reference across the system. Voltage levels are carefully managed to match the operational requirements of each component, and decoupling capacitors may be added during physical implementation to reduce voltage noise.

This connection ensures that the hardware functions smoothly with minimal interference, accurate readings, and safe power delivery. It also reflects the real-world constraints and considerations in building a robust embedded system for critical applications like cold storage monitoring.

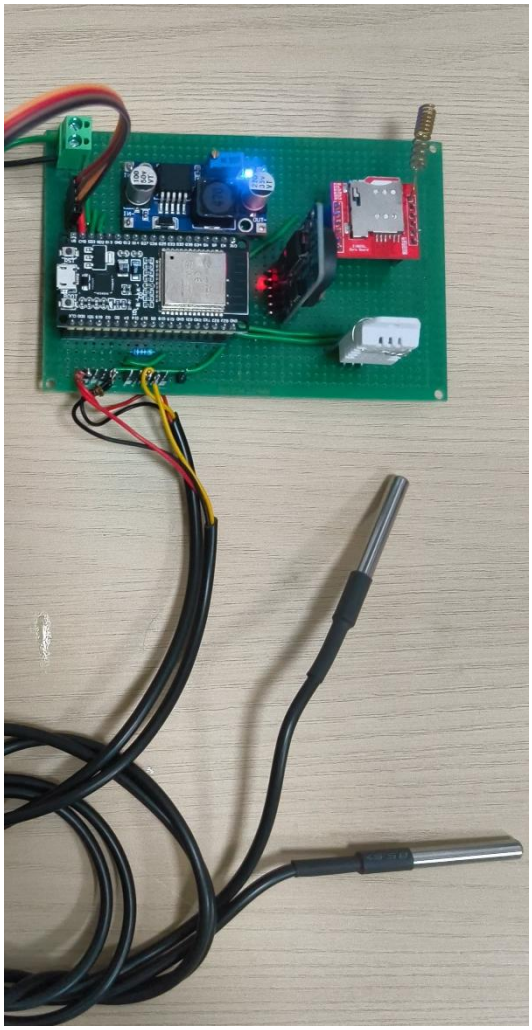


Fig: Data logging system

3.3 SOLAR POWER INTEGRATION

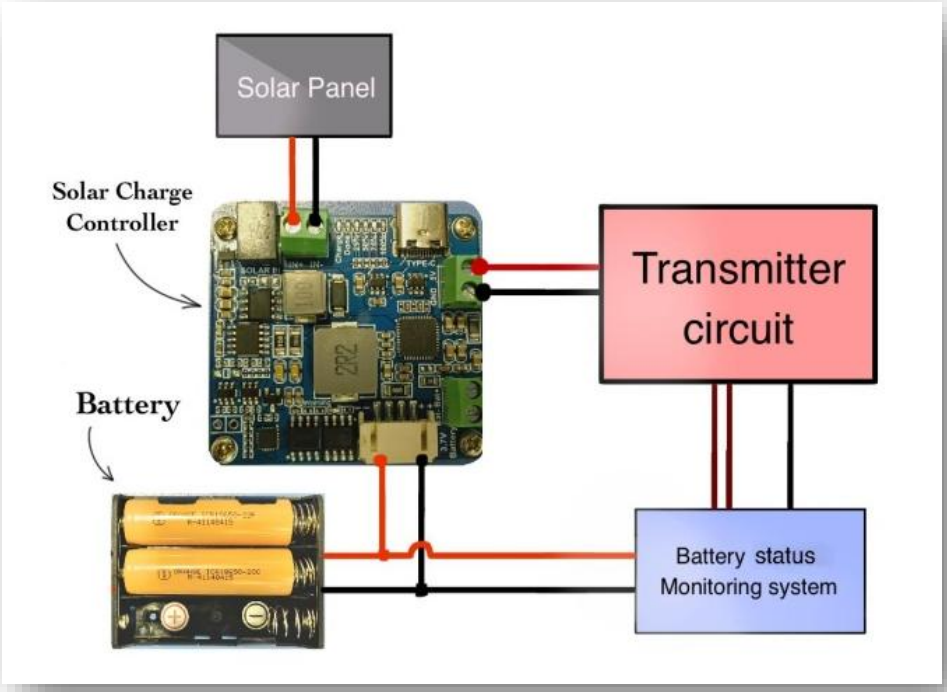


Fig: Connection diagram of Power supply

To power the transmitter, battery and solar panel is used. During sunshine hours, solar panel charges the battery as well as powers the transmitter node. During night time, the battery powers the transmitter node. For proper charging and supplying of power, a solar charge controller is employed. The solar charge controller has a feature called Maximum Power Point Tracking (MPPT) which maximises the energy output from the solar panel. The charge controller also has a build in buck-boost converter that maintains a stable 5V dc at its output. Connection diagram of the power supply is shown in fig. Following are the materials used in the power supply along with their specifications:

i) Solar panel (18V)

ii) Solar charge controller (Waveshare Solar power manager D)

Input: 6V-24V, Output: 5V/1A)

iii) Li-ion Battery (3.7V, 2200 mAh)

Fig: Testing of Solar Power

3.4 BATTERY MONITORING SYSTEM

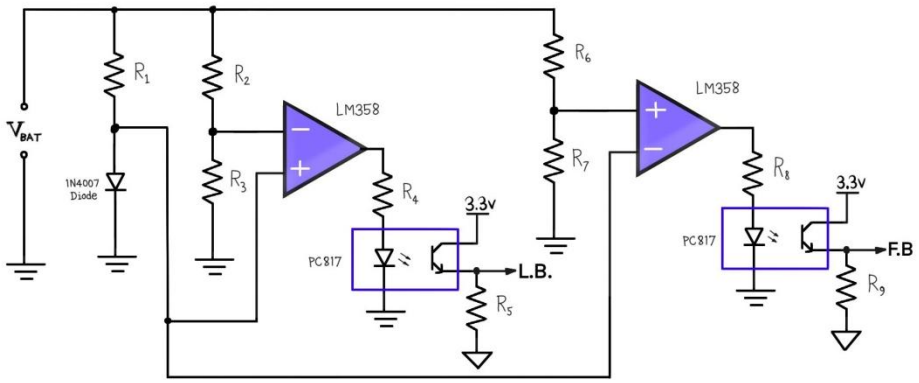


Fig: Battery monitoring circuit diagram

The system consists of two op-amps configured as comparators. A resistor and a diode areused for getting 0.7v which is used as a reference voltage and this voltage is given to each comparator, at non inverting input of the first comparator and at the inverting input of the second comparator. The other inputs of each comparator are connected to the output of separate voltage dividers. The voltage divider consisting of resistors R_2 and R_3 is designed in such a way that if battery voltage is 3.3v then output of the voltage divider will be 0.7v.

This way if voltage of the battery falls below 3.3v then the output of the first comparator becomes high which indicates low battery and sends this signal to the controller unit through an optocoupler network.

Similarly for indicating full battery, another voltage divider consisting of resistors R6 and R7 is designed in such a way that if battery voltage is 4v then output of the voltage divider becomes 0.7v. This way the second comparator outputs

high if voltage is above 4v. Then this signal is sent to controller unit through an optocoupler network. The reason that an optocoupler is employed is to isolate the ground of the controller unit with the ground of the battery. The table below shows the the state of F.B. and L.B. according to the battery voltage:

Battery voltage	F.B.	L.B.	Battery status
<3.3v	0	1	Battery is low
>3.3v and <4v	0	0	Battery is OK
>4v	1	0	Battery is full

Fig: Testing Battery monitoring system

3.5 MICROCONTROLLER CONFIGURATION

The ESP32 operates as the main controller and is programmed using the Arduino IDE. It is responsible for acquiring temperature data from three different sensors — two DS18B20 digital temperature sensors and one DHT22 sensor. The DS18B20 sensors are connected to GPIO18 and GPIO14 respectively using the OneWire protocol, while the DHT22 sensor is connected to GPIO23. These sensors allow precise monitoring of the storage environment at multiple locations.

To ensure accurate time-stamping of each reading, the system uses a DS3231 Real-Time Clock (RTC) module connected to the ESP32 via I2C interface (SDA on GPIO21 and SCL on GPIO22). The RTC ensures the device maintains accurate time even during power loss.

Battery Monitoring Circuit

Battery status is monitored using a custom-built analog circuit involving a comparator, diode, optocoupler, and resistors. The outputs of this circuit are fed into two digital input pins of the ESP32: GPIO12 and GPIO13. These inputs are used to determine whether the battery is in a full, low, or normal state by encoding them into a simple battery status code (1 = low, 2 = medium, 3 = full), which is also logged and transmitted.

Data Collection and Timing:

The code is configured to collect temperature readings every 20 seconds. These readings are stored temporarily in a structured array containing time-stamps, temperature values, and battery status. After three sets of readings (i.e., one minute), the data is compiled into a single JSON object and prepared for upload to the cloud.

Wireless Communication via SIM800L:

The system uses the SIM800L GSM module for wireless communication. The module is powered by a dedicated LM2596 buck converter, which steps down the input voltage to a stable 4.2V required by the SIM800L. Communication between the ESP32 and SIM800L occurs over UART2 (TX on GPIO17, RX on GPIO16). The ESP32 sends AT commands to the SIM800L to establish a GPRS connection, format the data, and upload it to the ThingSpeak cloud server using the HTTP protocol.

Data Upload and Storage:

Data is sent in bulk format to ThingSpeak every minute. The upload includes three timestamped readings, each with values from all three sensors and the battery code. The use of bulk upload reduces power and bandwidth usage and allows ThingSpeak to efficiently log and graph the sensor values over time. ThingSpeak's interface also supports long-term storage and visualization of environmental conditions, making it suitable for monitoring and diagnostics in cold storage applications.

Code:

```
#include <OneWire.h>
#include <DallasTemperature.h>
#include <DHT.h>
#include <Wire.h>
#include "RTClib.h"

#define DS1_pin 18   // 1s DS18B20 pin
#define DS2_pin 14   // 2nd DS18B20 pin
#define DHT_PIN 23       // DHT22 pin
#define DHT_TYPE DHT22
```

```
#define fullBatt 12
#define lowBatt 13

HardwareSerial sim800(2); // SIM800L on UART2 (RX = 16, TX = 17)
OneWire oneWire1(DS1_pin);
OneWire oneWire2(DS2_pin);
DallasTemperature sensor1(&oneWire1);
DallasTemperature sensor2(&oneWire2);
DHT dht(DHT_PIN, DHT_TYPE);
RTC_DS3231 rtc;

const char* APN = "airtelgprs.com"; // "*"" means pointer. it points to a const
character or strinng
const char* API_KEY = "UW3X9ZE4NVMF0EF3";
const char* CHANNEL_ID = "3010755";


// Data Storage
struct TempData{   //TempData is an obj that has the following structure
  String timestamp;
  float t1, t2, t3;
  int BatteryLevel;
};
TempDatareadings[3]; //readings is an array of 4elements each having structure
as TempData
int readingIndex = 0;

//func declarations, they tell the compiler that these funcs exist and will be used
void collectTemperature(); //void so takes no i/p parameters but collectes temp
data
void sendAT(String cmd);// returns nothing but takes string as i/p i.e AT cmds
bool waitForResponse(String target, int timeout);//gives true if target
string(could be anything, eg: OK) is received within 'timeout' time
String buildJSON();//returns a string (JSON obj), takes no i/p
void sendToThingSpeak();
String getBatteryStatus(); //returns a string
```

```
// === Setup ===
void setup() {
  Serial.begin(115200);
  sim800.begin(9600, SERIAL_8N1, 16, 17); //8 data bits, 1stop bit
  sensor1.begin();
  sensor2.begin();
  dht.begin();
  rtc.begin();
  pinMode(fullBatt, INPUT);
  pinMode(lowBatt, INPUT);
  delay(1000);
  Serial.println("Setup complete");
}

// === Main Loop ===
int lastSecondChecked = -1;

void loop() {
  DateTime now = rtc.now();
  int currentSecond = now.second();
  int currentMinute = now.minute();

  // send only after 5mins i.e. multiple of 5's (0, 20, 40)
   if ((currentSecond % 20 == 0) && (currentSecond != lastSecondChecked)
&&readingIndex< 12) {
    collectTemperature();
    readingIndex++;
    lastSecondChecked = currentSecond;
  }
  if (readingIndex>= 3) {
    Serial.println("Sending batch to ThingSpeak...");
    sendToThingSpeak();
    readingIndex = 0;
  }

  delay(1000);
}
```

```
// Collect temperature and timestamp
void collectTemperature() {
  sensor1.requestTemperatures();
  float t1 = sensor1.getTempCByIndex(0);
  sensor2.requestTemperatures();
  float t2 = sensor2.getTempCByIndex(0);
  float t3 = dht.readTemperature();

  if (isnan(t3)) {
    Serial.println("DHT read failed!");
    t3 = -999.0;
  }
  int batteryCode;
  if (digitalRead(fullBatt) == HIGH) {
    batteryCode = 3;
  } else if (digitalRead(lowBatt) == HIGH) {
    batteryCode = 1;
  } else {
    batteryCode = 2;
  }
  DateTime now = rtc.now();
  char ts[25];
  sprintf(ts, "%04d-%02d-%02dT%02d:%02d:%02dZ",
        now.year(), now.month(), now.day(),
        now.hour(), now.minute(), now.second());

  readings[readingIndex].timestamp = String(ts);
  readings[readingIndex].t1 = t1;
  readings[readingIndex].t2 = t2;
  readings[readingIndex].t3 = t3;
  readings[readingIndex].batteryCode = batteryCode;

  Serial.println("Stored Reading " + String(readingIndex) +
              ": DS1=" + String(t1) +
              "°C, DS2=" + String(t2) +
              "°C, DHT22=" + String(t3) +
```

```
            "°C, BatteryCode=" + String(batteryCode) +
            " at " + String(ts));
}
// === Send to ThingSpeak via SIM800L ===
void sendToThingSpeak() {
  String payload = buildJSON();

  sendAT("AT");
  sendAT("AT+SAPBR=3,1,\"CONTYPE\",\"GPRS\"");
  sendAT("AT+SAPBR=3,1,\"APN\",\"" + String(APN) + "\"");
  sendAT("AT+SAPBR=1,1");
  delay(3000);
  sendAT("AT+SAPBR=2,1");

  sendAT("AT+HTTPINIT");
  sendAT("AT+HTTPPARA=\"CID\",1");
     sendAT("AT+HTTPPARA=\"URL\",\"http://api.thingspeak.com/channels/"    +
String(CHANNEL_ID) + "/bulk_update.json\"");
  sendAT("AT+HTTPPARA=\"CONTENT\",\"application/json\"");

  sim800.println("AT+HTTPDATA=" + String(payload.length()) + ",10000");
  delay(100);
  if (waitForResponse("DOWNLOAD", 5000)) {
    sim800.print(payload);
    delay(3000);
    sendAT("AT+HTTPACTION=1");
    delay(6000);
  }
  sendAT("AT+HTTPTERM");
  //sendAT("AT+SAPBR=0,1");

  delay(20000);
// === Build JSON for bulk upload ===
String buildJSON() {
  String json = "{\"write_api_key\":\"" + String(API_KEY) + "\",\"updates\":[";
  for (int i = 0; i< 3; i++) {
    json += "{\"created_at\":\"" + readings[i].timestamp + "\",";
```

```
    json += "\"field1\":" + String(readings[i].t1, 2) + ",";
    json += "\"field2\":" + String(readings[i].t2, 2) + ",";
    json += "\"field3\":" + String(readings[i].t3, 2) + ",";
    json += "\"field4\":" + String(readings[i].batteryCode) + "}";
    if (i< 2) json += ",";
  }
  json += "]}";
  return json;
}

// Send AT and print response
void sendAT(String cmd) {
  sim800.println(cmd);
  delay(500);
  while (sim800.available()) {
    Serial.write(sim800.read());
  }
}

// Wait for a keyword in SIM800L response
bool waitForResponse(String target, int timeout) {
  unsigned long start = millis();
  String response = "";
  while (millis() - start < timeout) {
    while (sim800.available()) {
      char c = sim800.read(); //reads one character from sim800's reply and adds
it to response strinng
      response += c; //one ch at a time because sim800 sends data byte by byte
and not full reply
      if (response.indexOf(target) != -1) return true;
    }
  }
  return false;
```

3.6 DATA TRANSMISSION TO CLOUD

In the proposed system, the data collected from the temperature sensors (two DS18B20 and one DHT22) and the battery level indicators is transmitted to the cloud using the SIM800L GSM module. The ESP32 microcontroller communicates with the SIM800L via UART protocol (using GPIO16 and GPIO17) to establish a GPRS connection through the Airtel mobile network, using the APN "airtelgprs.com". After initializing the GPRS session, the ESP32 formats the collected data into a JSON object that includes timestamps and readings from all sensors. This JSON payload is then uploaded to the ThingSpeak cloud platform using HTTP POST requests, following the bulk update format supported by ThingSpeak. The SIM800L executes a sequence of AT commands to handle this communication process, ensuring the data is sent securely and reliably. The ThingSpeak server stores the uploaded data and provides visualization through individual graphs for each sensor value and battery status. This enables continuous remote monitoring of environmental conditions in the cold storage system, even in locations without Wi-Fi or broadband connectivity. The system also uses a response-checking function to ensure data transmission was successful before terminating the HTTP session.

3.7 Final Mounting And Setup

After successful testing and verification of individual components and the integrated system, the complete setup was securely mounted onto a hardware base for deployment. All sensors, including the DS18B20 and DHT22, were positioned in appropriate locations to ensure accurate environmental readings. The ESP32 microcontroller, SIM800L GSM module, and associated circuitry were enclosed in a protective casing to safeguard against dust, moisture, and physical damage. Special attention was given to proper cable management and soldering connections to prevent loose contacts and ensure system stability. The power supply, including the solar charging unit and battery indicators, was installed in a way that allowed easy access and visibility of battery status.

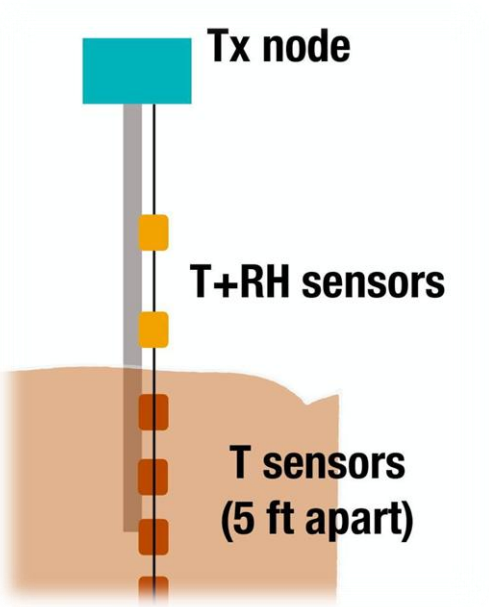


Fig: Transmitter node with sensors

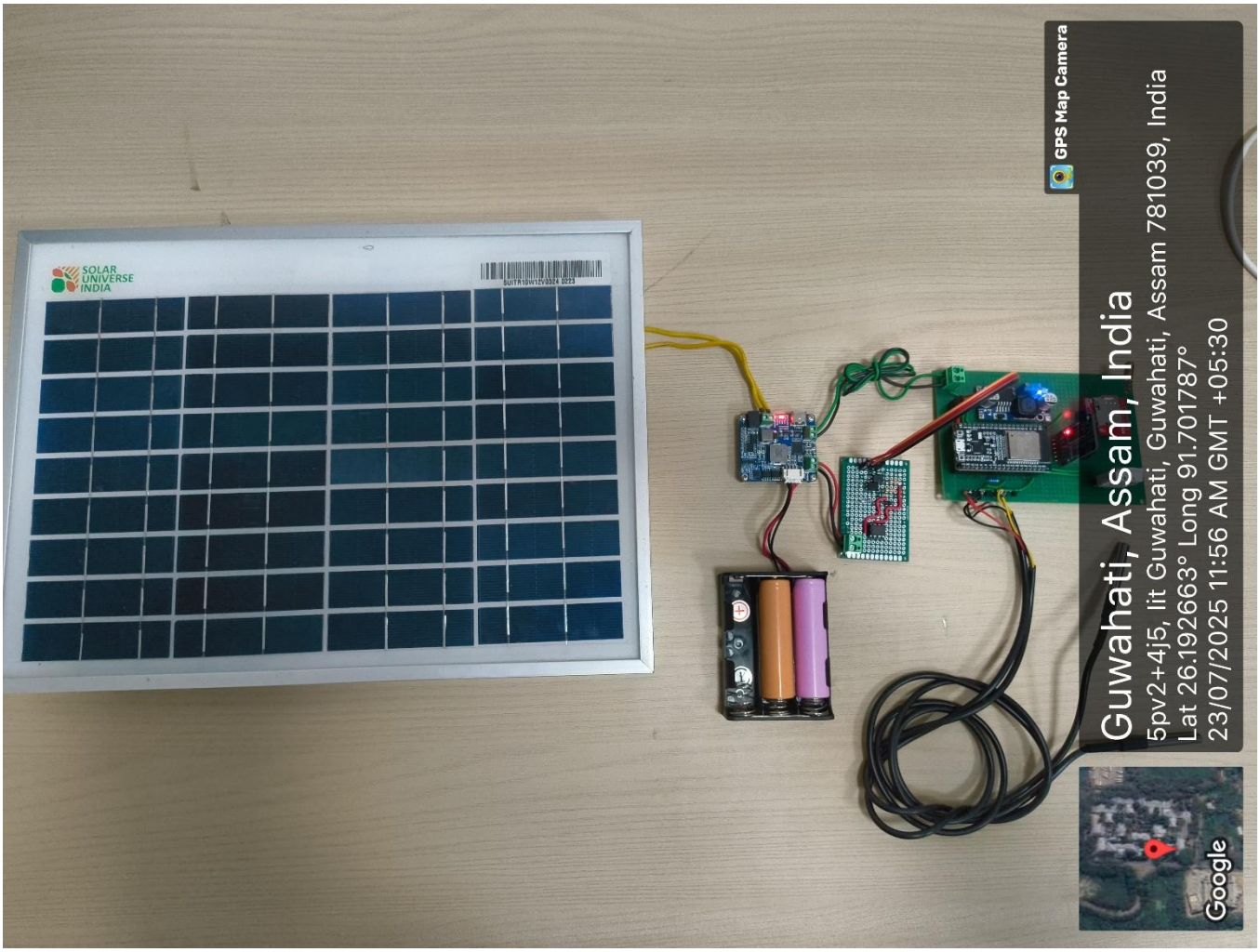


Fig: Data logging system with integrated solar power

CHAPTER 4
RESULTS AND DISCUSSION

The temperature data logging system was successfully assembled and tested to evaluate its performance in capturing temperature, humidity, and battery voltage status, and transmitting this data to the cloud. The system incorporated two DS18B20 digital temperature sensors, one DHT22 sensor, and a custom-built battery monitoring circuit, all interfaced with an ESP32 microcontroller.

4.1 SENSOR PERFORMANCE

The system was designed to collect environmental data every 20 seconds and transmit it to the ThingSpeak cloud platform in batches every one minute.

- The DS18B20 sensors recorded temperature values with high stability and precision. The sensors responded accurately to applied temperature changes, and their output was logged with time-stamps for each reading.
- The DHT22 sensor provided real-time measurements of ambient temperature and humidity. During testing, the temperature readings remained consistent with the applied environmental changes, and the humidity values followed expected variations.

All sensor data were processed by the ESP32 and successfully formatted into JSON payloads. The data fields displayed on the ThingSpeak platform allowed for continuous observation and trend analysis.

Fig: Graph of recorded data in ThingSpeak

The battery monitoring circuit, based on comparator logic, provided a simplified yet effective method to track battery voltage levels. The ESP32 read the battery status via digital input pins and mapped the condition to a discrete code, which was then uploaded to ThingSpeak as a separate data field.
The values were defined as follows:

- 1 — Battery Low (below 3.3V)
- 2 — Battery OK (between 3.3V and 4.0V)
- 3 — Battery Full (above 4.0V)

To validate this functionality, various voltages were applied manually during testing:

- When the voltage dropped below 3.3V, the system correctly registered a value of 1.

- At intermediate voltages, it reported 2, indicating a stable level.
- Above 4.0V, the ESP32 recorded a battery status of 3, confirming full charge conditions.

These values were consistently updated on ThingSpeak, proving the reliability of the battery monitoring system.

4.2 Overall System Validation

The device was tested under controlled conditions by applying temperature changes to the sensors and varying the battery voltage. All components worked in synchronization:

- The sensors recorded environmental data accurately.
- The ESP32 handled data collection, formatting, and transmission reliably.
- The SIM800L consistently sent data packets to the ThingSpeak server.
- The battery monitor reflected voltage changes in real time.

Overall, the system demonstrated its intended functionality effectively, validating the design approach and integration of hardware and software components.

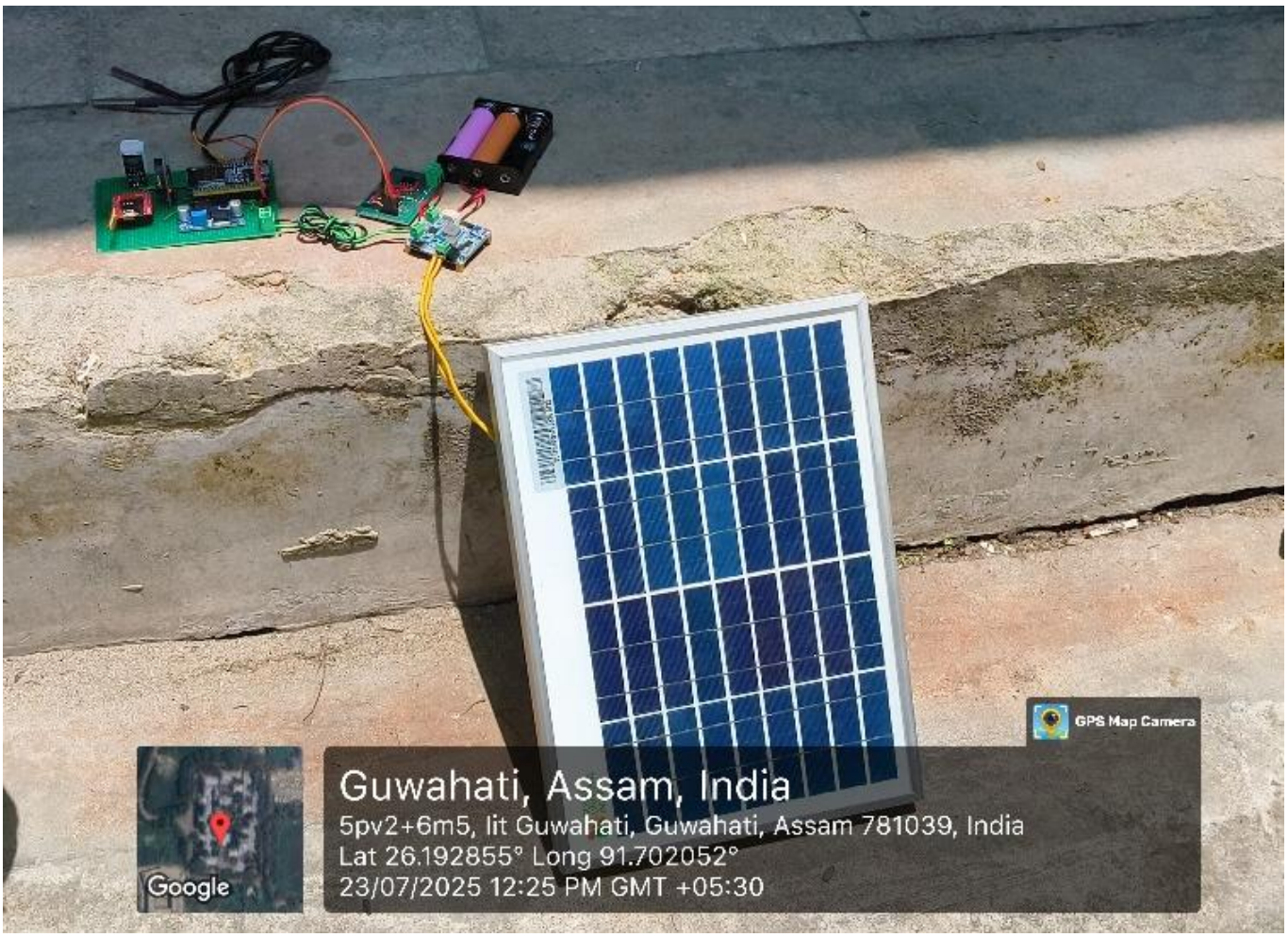


Fig: System during test

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

This project focused on the development of a real-time cold storage monitoring system using the ESP32 microcontroller, DS18B20 digital temperature sensors, DHT22 sensor, SIM800L GSM module, and a battery monitoring circuit. The system was designed to continuously monitor environmental conditions, particularly temperature, and send the readings in JSON format to the ThingSpeak cloud platform every minute. A key feature was the use of multiple sensors to ensure reliable data acquisition and the integration of a low-cost GSM module to eliminate the need for Wi-Fi infrastructure, making the system suitable for remote and rural areas.

The project met its main objectives, including accurate temperature measurement, periodic data logging, cloud integration, and basic battery status detection using a comparator and optocoupler circuit. The cloud storage and visualization features of ThingSpeak allowed for easy access and long-term monitoring of data, with automatic graph plotting for better analysis. The system proves beneficial for cold storage facilities where consistent temperature maintenance is essential for preserving agricultural and perishable products. Overall, the implementation was cost-effective, scalable, and well-suited for real-world deployment

5.2 LIMITATIONS

Although the project was successfully completed, certain limitations were encountered during the development and testing phases:

- GSM Connectivity Issues: The SIM800L module was sensitive to power fluctuations and network signal strength, which occasionally led to transmission failures or delays. This affected the reliability of real-time data updates.
- Limited Battery Monitoring Accuracy: The battery level detection was implemented using a simple comparator-based system with discrete level indication (Low, Medium, High). However, this method does not provide exact voltage or battery health status.
- Sensor Accuracy and Calibration: While DS18B20 sensors are accurate, external factors like wire resistance, electromagnetic interference, and improper placement can cause minor variations in readings.
- Cloud Service Constraints: The ThingSpeak free plan has a limit of data uploads (once every 15 seconds, and a daily cap), which restricts the frequency and resolution of monitoring, especially if more sensors or parameters are added.
- Hardware Protection: The system lacked protection against overvoltage, reverse polarity, or physical damage, which could impact long-term use in industrial or outdoor environments.

5.3FUTURE ENHANCEMENTS

To improve the functionality, accuracy, and robustness of the system, several future enhancements are proposed:

- Advanced Battery Monitoring: Replace the comparator-based method with an analog voltage divider and ADC reading via ESP32 to display real-time battery voltage and estimate battery health more precisely.
- Solar Power Integration: Include a compact solar charging system with battery management to make the system energy-independent, especially in remote areas without stable electricity.
- Use of MQTT Protocol: Implement the MQTT protocol instead of HTTP for faster, more secure, and reliable data communication. MQTT is lightweight and well-suited for IoT applications.
- Mobile App and Web Dashboard: Develop a user-friendly mobile application or web portal for real-time data visualization, custom alerts, notifications (SMS/email), and remote control of storage systems.
- Expanded Sensing Capabilities: Add sensors for humidity, light intensity, gas leakage, or door status to ensure comprehensive monitoring. These can further improve storage condition analysis.
- Data Backup and Logging: Store data locally on an SD card in case of network failure, ensuring no data loss and allowing retrospective analysis.
- Machine Learning for Prediction: Integrate basic machine learning algorithms for anomaly detection or predictive maintenance based on historical data trends.

REFERENCE

1. Dallas Semiconductor. “DS18B20: Programmable Resolution 1-Wire Digital Thermometer.” Datasheet
2. Adafruit Industries. “DHT22 Temperature-Humidity Sensor.” Product Documentation
3. Espressif Systems. “ESP32-WROOM-32 Technical Reference Manual.” Online Resource
4. SIMCom Wireless Solutions. “SIM800L GSM/GPRS Module AT Command Manual.” AT Commands Manual
5. ThingSpeak™ Platform Documentation. MATLAB IoT Platform for Data Collection and Visualization. Official Website
6. RTClib Library for DS3231. Adafruit RTClib GitHub Repository.
7. Waveshare. “Solar Power Manager D.” Product Datasheet and Features. Product Page
8. Banerjee, S., & Patel, M. (2018). *IoT-based Smart Monitoring System for Cold Storage Applications*. International Journal of Engineering Trends and Technology (IJETT), 58(2), 98–104.